LAPORAN PRATIKUM STRUKTUR DATA

ADT

Disusun Oleh:

Shofwatul Hasanah Sarma

NIM 2511533015

Dosen Pengampu: Wahyudi. DR, S.T.M.T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2025

KATA PENGANTAR

Puji dan syukur kita panjatkan kehadirat allah yang maha Esa karena telah memberikan nikmat sehat jasmani dan rohani sehingga saya bisa menyelesaikan laporan ini. Di dalam laporan ini, terdapat hasil pratikum, pembahasan serta kesimpulan yang saya susun secara sistematis. Saya sebagai penulis sekaligus penyusun laporan ini sangat berterimakasih kepada bapak Wahyudi selaku dosen dalam mata kuliah pratikum Struktur Data Departemen Informatika, Universitas Andalas, yang telah menuntun dan mengajarkan saya mata kuliah ini.

Dalam penulisan laporan pratikum ini, saya mendapatkan suatu pelajaran baik serta mendapatkan pengetahuan tentang pratikum tersebut. Saya sebagai penulis dalam laporan ini masih dalam proses belajar, baik dalam tata penulisan laporan yang benar maupun dalam memahami penyajikan laporan yang sesuai kaidah. Untuk itu saya berharap mendapatkan saran saran yang membangun sehingga dapat meningkatkan pengetahuan saya dan dapat membuat isi laporan yang lebih baik, mendidik dan dapat dipertanggungjawabkan.

Padang, 2025

Penulis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I 3

PENDAHULUAN 3

1.1 Latar Belakang 3

1.2 Tujuan Pratikum 3

1.3 Manfaat Pratikum 3

BAB II 4

PEMBAHASAN 4

2.1 Pengertian 4

2.1.1 Pengertian ADT 4

2.2 Pratikum 4

2.2.1 Alat dan Bahan 4

2.2.2 Langkah langkah 4

BAB III 12

KESIMPULAN 12

3.1 Ringkasan Hasil Pratikum 12

3.2 Saran Pengembangan 12

DAFTAR PUSTAKA 13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, pengelolaan data yang efisien menjadi hal yang sangat penting. Struktur data berperan dalam mengatur, menyimpan dan mengakses data agar lebih optimal. Salah satu konsep dasar dalam struktur data Adalah Abstract Data Type (ADT), yang berfokus pada apa yang dilakukan suatu struktur data tanpa mempedulikan bagaimana implementasinya.

Praktikum ini dilakukan untuk membantu mahasiswa memahami konsep ADT secara teoritis dan praktis, serta bagaimana penerapannya dalam pemograman.

1.2 Tujuan Pratikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar Abstract Data Type (ADT)
2. Mempelajari perbedaan antara konsep (abstraksi) dan implementasi struktur data
3. Mengimplementasikan ADT menggunakan Bahasa pemograman
4. Melatih kemampuan berpikir logis dan terstuktur dalam pengolahan data

1.3 Manfaat Pratikum

Adapun manfaat dari praktikum ADT ini adalah sebagai berikut:

1. Meningkatkan pemahaman tentang konsep abstraksi dalam pempgraman
2. Membantu dalam merancang program yang lebih modular dan terstruktur
3. Mempermudah pengembangan dan pemeliharaan kode program
4. Menjadi dasar untuk mempelajari struktur data yang lebih kompleks seperti stack, queue dan tree

BAB II

PEMBAHASAN

4.1 Pengertian

4.1.1 Pengertian ADT

ADT Adalah suatu model atau konsep struktur data yang hanya mendefinisikan operasi operasi yang dapat dilakukan terhadap data tanpa menjelaskan bagaimana data tersebut diimplementasikan.

ADT berfokus pada interface (apa yang bisa dilakukan), bukan implementasi (bagaimana cara melakukannya)

Contoh:

ADT stack memiliki operasi seperti:

- Push (menambah data)
- Pop (menghapus data)
- Peek (melihat data teratas

  Namun, implementasinya bisa menggunakan array atau linkes list.

4.2 Pratikum

4.2.1 Alat dan Bahan

a. Komputer / Laptop

   Digunakan untuk mengaplikasikan aplikasi eclipse

b. Aplikasi eclipse

c. Paket data atau jaringan yang stabil

4.2.2 Langkah Langkah

```
5. private int hh; // 0..23
6.                    private int mm; // 0..59
7.                    private int ss; // 0..59
```

```java

                    // ---Validator ---
                        public static boolean isValid (int h, int m, int s) {
                            return (0 <= h && h <= 23) && ( 0 <= m && m <= 59) && ( 0 <= s &&  s <= 59);
                        }
                        // ---Konstruktor/Kreator---
                        public Jam_2511533015 (int h, int m, int s) {
                            this.hh = h; this.mm = m; this.ss = s;
                        }
                        // ---Selektor---
                        public int getHour () {return hh;}
                        public int getMinute () {return mm;}
                        public int getSecond () {return ss;}
```

artinya:

1. Package dan kelas

   package pekan1_2511533015;

   public final class Jam_2511533015 {

   - Package pekan1_2511533015:

     Kode ini masuk ke dalam paket (folder) Bernama pekan1_2511533015

   - Public final class Jam_2511533015

     Membuat kelas Bernama Jam_2511533015

     Final artinya kelas ini tidak bisa diturunkan

2. Atribut (variable)

   private int hh; // 0..23

   private int mm; // 0..59

   private int ss; // 0..59

   - hh: menyimpan jam (hour) dari 0-23
   - mm: menyimpan menit (minute) dari 0-59
   - ss: menyimpan detik (second) dari 0-59
   - private: hanya bisa diakses dari dalam kelas

3. validator

   public static boolean isValid (int h, int m, int s) {

```
    return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0
<= s && s <= 59);
}
```

- method ini untuk mengecek apakah waktu valid
- static: bisa dipanggil tanpa membuat objek
- mengembalikan true jika:

  jika antara 0-23

  menit 0-59

  detik 0-59

  contoh:

```
Jam_2511533015.isValid(12, 30, 45); // true
Jam_2511533015.isValid(25, 10, 10); // false
```

4. konstruktor

```
public Jam_2511533015 (int h, int m, int s) {
    this.hh = h;
    this.mm = m;
    this.ss = s;
}
```

- digunakan untuk membuat objek jam
- this.hh= h : mengisi nilai jam dari parameter

  contoh:

```
Jam_2511533015 jam1 = new Jam_2511533015(10, 20, 30);
```

5. selector (getter)

```
public int getHour () {return hh;}
public int getMinute () {return mm;}
public int getSecond () {return ss;}
```

- digunakan untuk mengambil nilai atribut
- karena atribut private, harus lewat getter

  contoh:

```
System.out.println(jam1.getHour());   // ambil jam
System.out.println(jam1.getMinute()); // ambil menit
```

System.out.println(jam1.getSecond()); // ambil detik

```
// ---Mutator---
                  public  void setHour(int h) {this.hh = h; }
                  public void setMinute(int m) {this.mm = m;}
                  public void setSecond(int s) {this.ss = s;}
                  // ---Konversi===
                  public int toSeconds () {return hh * 3600 + mm * 60 + ss;}
                  public static Jam_2511533015 fromSeconds (int total) {
                        if (total < 0) throw new IllegalArgumentException
("detik negatif");
                        total %= 24 * 3600; //wrap 24 jam
                        int h = total /3600 ; total %= 3600;
                        int m = total /60 ; int s = total % 60;
                        return new Jam_2511533015 (h,m,s);
                        }
                  //---Relasional---
                  public int compareTo (Jam_2511533015 other) {return
Integer.compare(this.toSeconds (), other.toSeconds ());}
                  public boolean equals (Object o) {
                        if (!(o instanceof Jam_2511533015 j)) return false;
                        return hh == j.hh && mm == j.mm && ss == j.ss;
                  }
                  public int hashCode () {return
java.util.Objects.hash(hh,mm,ss);
                  }
                  //--- Aritmatika---
                  public Jam_2511533015 plus (Jam_2511533015 other) {return
fromSeconds (this.toSeconds () + other.toSeconds());}
                  public Jam_2511533015 minus (Jam_2511533015 other) {return
fromSeconds (Math.floorMod (this.toSeconds() - other.toSeconds (),
24*3600));}
                  public Jam_2511533015 nextSecond () {return fromSeconds
(this.toSeconds() + 1);}
                  public Jam_2511533015 nextNSeconds (int n) { return
fromSeconds (this.toSeconds () + Math.max(0, n));}
                  public Jam_2511533015 prevSecond () {return fromSeconds
(Math.floorMod(this.toSeconds () - 1,  24*3600 ));}
                  public Jam_2511533015 prevNSeconds (int n) {return
fromSeconds (Math.floorMod(this.toSeconds() - Math.max(0, n),  24*3600));}

                  public static int durasiDetik (Jam_2511533015 jaw,
Jam_2511533015 jakh) {return jakh.toSeconds() -jaw.toSeconds();}
                  //overriding ke string
                  public String toString() { return
String.format("%02d:%02d:%02d", hh, mm, ss);}
```

```
    }
```

Artinya:

6. mutator

   ```
   public void setHour(int h) { this.hh = h; }
   public void setMinute(int m) { this.mm = m; }
   public void setSecond(int s) { this.ss = s; }
   ```

- fungsi:

  untuk mengubah nilai jam, menit, detik

  disebut mutator karena mengubah state objek

  contoh:

  jam1.setHour(12);

  jam1.setMinute(30);

  jam1.setSecond(45);

7. konversi ke detik

   ```
   public int toSeconds() {
       return hh * 3600 + mm * 60 + ss;
   }
   ```

- fungsi:

  mengubah waktu jadi total detik

  contoh:

  01:01:01 = 1x3600 + 1x60 + 1 = 3661 detik

8. konversi dari detik ke jam

   ```
   public static Jam_2511533015 fromSeconds(int total) {
       if (total < 0) throw new IllegalArgumentException("detik negatif");

       total %= 24 * 3600; // wrap 24 jam

       int h = total / 3600;
       total %= 3600;
   ```

```
        int m = total / 60;
        int s = total % 60;

        return new Jam_2511533015(h, m, s);
    }
```

- fungsi:

  mengubah detik = format jam

  wrap 24 jam = kalua lebih dari 24 jam, diputar ulang

  contoh:

  9000 detik = jadi jam normal (tidak lewat 24 jam)

9. relasional

   COMPARE TO

```
   public int compareTo(Jam_2511533015 other) {
       return Integer.compare(this.toSeconds(), other.toSeconds());
   }
```

- fungsi

  membandingkan dua waktu

  hasil:

  negative = lebih kecil

  0 = sama

  Positif = lebih besar

  EQUALS

```
  public boolean equals(Object o) {
      if (!(o instanceof Jam_2511533015 j)) return false;
      return hh == j.hh && mm == j.mm && ss == j.ss;
  }
```

- fungsi:

  mengecek apakah dua objek jam sama persis

  HASHCODE

```
  public int hashCode() {
      return java.util.Objects.hash(hh, mm, ss);
```

```
}
```

- fungsi

  digunakan untuk sruktur data seperti hashmap/ hashset

  OPERASI ARITMATIKA

  PENJUMLAHAN WAKTU

```
public Jam_2511533015 plus(Jam_2511533015 other) {
    return fromSeconds(this.toSeconds() + other.toSeconds());
}
```

- menambahkan dua waktu

  PENGURANGAN WAKTU

```
public Jam_2511533015 minus(Jam_2511533015 other) {
    return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(),
24*3600));
}
```

- mengurangi waktu (hasil tetap valid 0-24 jam)

10. operasi waktu

    TAMBAH 1 DETIK

```
public Jam_2511533015 nextSecond() {
    return fromSeconds(this.toSeconds() + 1);
}
```

    TAMBAH N DETIK

```
public Jam_2511533015 nextNSeconds(int n) {
    return fromSeconds(this.toSeconds() + Math.max(0, n));
}
```

    KURANG 1 DETIK

```
public Jam_2511533015 prevSecond() {
    return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600));
}
```

    KURANG N DETIK

```
public Jam_2511533015 prevNSeconds(int n) {
```

```
    return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n),
24*3600));
}
```

- semua pakai wrap 24 jam (tidak keluar dari format waktu normal)

11. durasi antar waktu

```
public static int durasiDetik(Jam_2511533015 jaw, Jam_2511533015
jakh) {
    return jakh.toSeconds() - jaw.toSeconds();
}
```

- fungsi

  mengubah onjek jadi format string:

  HH:MM:SS

  Contoh:

  Jam_2511533015 j = new Jam_2511533015(5, 7, 9);

  System.out.println(j);

  Output:

  05:07:09

BAB III

KESIMPULAN

4.1 Ringkasan Hasil Pratikum

Praktikum ini membahas konsep dasar ADT sebagai bagian penting dalam struktur data. ADT memungkinkan pemograman untuk memisahkan antara definisi dam implementasi suatu struktur data. Dengan memahami ADT , mahasiswa dapat membuat program yang lebih fleksibel, modular dan mudah dikembangkan. Pratikum ini juga memberikan pengalaman langsung dalam mengimplementasikan ADT menggunkan Bahasa pemograman.

4.2 Saran Pengembangan

1. Mengembangkan implementasi ADT ke berbagai struktur data lain seperti stack, queue dan tree
2. Mencoba berbagai metode implementasi (array vs linked list) untuk membandingkan efisiensi
3. Mengintegrasikan ADT dalam proyek aplikasi nyata
4. Menambahkan analisis kompleksitas waktu (time complexity) pada setiap operasi ADT
5. Mengunakan Bahasa pemrograman berbeda untuk memperluas pemahaman konsep

DAFTAR PUSTAKA

- Mark Allen Weiss. 2014. *Data Structures and Algorithm Analysis in Java*. Pearson Education.
- Robert Lafore. 2003. *Data Structures and Algorithms in Java*. Sams Publishing.
- Herbert Schildt. 2018. *Java: The Complete Reference*. McGraw-Hill Education.
- Oracle. *Java Documentation*. Diakses dari: https://docs.oracle.com
- GeeksforGeeks. *Abstract Data Type (ADT)*. Diakses dari: https://www.geeksforgeeks.org

Tugas Pekan 1

Soal

Tugas Praktikum Pekan 1

- Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk
- Contoh: Avanza, 2020, 1300, 150000000, Toyota
- Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, Selektor dan mutator
- Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan nim

Jawaban

1. Kelas pertama

```
package tugaspekan1_2511533015;

public class Mobil_2511533015 {

    // atribut
    String nama, merk;
    int tahun, cc, harga;

        // constructor
        public Mobil_2511533015(String nama, int tahun, int cc, int harga, String merk) {
            this.nama = nama;
            this.tahun = tahun;
            this.cc = cc;
            this.harga = harga;
            this.merk = merk;
        }

        // toString
        @Override
        public String toString() {
            return nama + " | " + tahun + " | " + cc + "cc | Rp" + harga + " | " + merk;
        }

        // GETTER
```

```java
        public String getNama() { return nama; }
        public int getCc() { return cc; }
        public int getHarga() { return harga; }
        public int getTahun() { return tahun; }
        public String getMerk() { return merk; }

        // SETTER
        public void setNama(String nama) { this.nama = nama; }
        public void setTahun(int tahun) { this.tahun = tahun; }
        public void setCc(int cc) { this.cc = cc; }
        public void setHarga(int harga) { this.harga = harga; }
        public void setMerk(String merk) { this.merk = merk; }

        // ARRAY GARASI
        private static int length = 0;
        private static Mobil_2511533015[] garasi = new Mobil_2511533015[0];

        // TAMBAH MOBIL
        public static void tambahMobil(Mobil_2511533015 mobil) {
            length++;
            Mobil_2511533015[] temp = new Mobil_2511533015[length];

            for (int i = 0; i < length - 1; i++) {
                temp[i] = garasi[i];
            }

            temp[length - 1] = mobil;
            garasi = temp;
        }

        // HAPUS MOBIL
        public static void hapusMobil(int index) {
            if (index < 0 || index >= length) {
                System.out.println("Index tidak valid!");
                return;
            }

            Mobil_2511533015[] temp = new Mobil_2511533015[length - 1];
            int j = 0;

            for (int i = 0; i < length; i++) {
                if (i != index) {
                    temp[j] = garasi[i];
                    j++;
                }
            }
```

```java

            length--;
            garasi = temp;
        }

        // TAMPILKAN MOBIL
        public static void tampilkanMobil() {
            if (length == 0) {
                System.out.println("Data kosong!");
                return;
            }

            for (int i = 0; i < length; i++) {
                System.out.println(i + ". " + garasi[i]);
            }
        }
    }
    }
```

Penjelasan:

a. Package dan class

package tugaspekan1_2511533015;

public class Mobil_2511533015 {

package tugaspekan1_2511533015

Menunjukkan bahwa class ini berada dalam folder / package tersebut.

public class mobil_2511533015

Mendefinisikan kelas Bernama mobil_2511533015 sebagai ADT mobil

b. Atribut

String nama, merk;

int tahun, cc, harga;

nama : nama mobil

merk : merk mobil

tahun : tahun produksi

cc : kapasitas mesin

harga : hatga mobil

Digunakan untuk menyimpan data mobil

c. Construktur

public Mobil_2511533015(String nama, int tahun, int cc, int harga, String merk)

digunakan untuk membuat objek mobil

this.nama = nama : mengisi atribut dengan nilai dari parameter

Contoh:

Mobil_2511533015 m = new Mobil_2511533015("Avanza", 2020, 1300, 150000000, "Toyota");

d. Method

```
@Override
public String toString() {
    return nama + " | " + tahun + " | " + cc + "cc | Rp" + harga + " | " + merk;
}
```

digunakan untuk menampilkan objek dalam bentuk string

otomatis dipanggil saat objek di print

Output:

Avanza | 2020 | 1300cc | Rp150000000 | Toyota

e. Getter

public String getNama() { return nama; }

digunakan untuk mengambil nilai atribut

karena atribut tidak diakses langsung

Contoh :

m.getNama();

f. Setter

public void setNama(String nama) { this.nama = nama; }

digunakan untuk mengubah nilai atribut

Contoh :

m.setNama("Innova");

g. Array garasi

private static int length = 0;

private static Mobil_2511533015[] garasi = new Mobil_2511533015[0];

garasi : array untuk menyimpan banyak objek mobil

length : jumlah data mobil saat ini

static : bisa digunakan tanpa membuat objek

h. Method tambah mobil

public static void tambahMobil(Mobil_2511533015 mobil)

menambahkan mobil ke dalam array

Proses :

Menambah ukuran array

Menyalin data lama ke array baru

Menambahkan data baru di akhir

i. Method hapus mobil

public static void hapusMobil(int index)

menghapus mobil berdasarkan index

Proses :

Cek index valid atau tidak

Salin semua data kecuali index yang dihapus

Kurangi jumlah data

j. Method tampilkan mobil

public static void tampilkanMobil()

menampilkan semua data mobil dalam array

Jika kosong:

Data kosong!

Jika ada :

0. Avanza | ...

1. Brio | ...

2. Kelas kedua

```
package tugaspekan1_2511533015;

public class Driver_2511533015 {
    public static void main(String[] args) {

        Mobil_2511533015 a = new Mobil_2511533015("Avanza", 2020, 1300, 150000000, "Toyota");
        Mobil_2511533015 b = new Mobil_2511533015("Brio", 2022, 1200, 170000000, "Honda");

```

```
11.            System.out.println("a = " + a);
12.            System.out.println("b = " + b);
13.
14.            Mobil_2511533015.tambahMobil(a);
15.            Mobil_2511533015.tambahMobil(b);
16.
17.            System.out.println("\nData Mobil:");
18.            Mobil_2511533015.tampilkanMobil();
19.
20.            Mobil_2511533015.hapusMobil(0);
21.
22.            System.out.println("\nSetelah dihapus:");
23.            Mobil_2511533015.tampilkanMobil();
24.        }
25.    }
```

Penjelasan:

a. Package dan class

   package tugaspekan1_2511533015;

   public class Driver_2511533015 {

   # package tugaspekan1_2511533015

   Menunjjukkan file ini berada dalam package tersebut

   # public class Driver_2511533015

   Class ini berfungsi sebagai program utama, untuk menjalankan program

b. Method main

   public static void main(String[] args) {

   # method utama yang pertama kali dijalankan oleh java

   # semua proses program dimulai dari sini

c. Membuat objek mobil

```
Mobil_2511533015 a = new Mobil_2511533015("Avanza", 2020,
1300, 150000000, "Toyota");
Mobil_2511533015 b = new Mobil_2511533015("Brio", 2022, 1200,
170000000, "Honda");

# membuat 2 objek mobil :
a : mobil Avanza
b : mobil brio
menggunakan constructor dari class Mobil_2511533015
```

d. Menampilkan objek

```
System.out.println("a = " + a);
System.out.println("b = " + b);
# menampilakan data mobil
# secara otomatis memanggil method toString()
Output :
a = Avanza | 2020 | 1300cc | Rp150000000 | Toyota
b = Brio | 2022 | 1200cc | Rp170000000 | Honda
```

e. Menambah ke garasi

```
Mobil_2511533015.tambahMobil(a);
Mobil_2511533015.tambahMobil(b);
# menambahkan objek mobil ke dalam array garasi
# menggunakan method static tambahMobil()
```

f. Menampilkan semua data mobil

```
System.out.println("\nData Mobil:");
```

Mobil_2511533015.tampilkanMobil();

menampilkan seluruh data mobil dalam garasi

\n : membuat baris baru

Output :

Data Mobil:

0. Avanza | ...

1. Brio | ...

g. Menghapus data mobil

Mobil_2511533015.hapusMobil(0);

menghapus mobil pada index ke -0 (mobil pertama)

dalam kasus ini : Avanza dihapus

h. Menampilkan setelah dihapus

System.out.println("\nSetelah dihapus:");

Mobil_2511533015.tampilkanMobil();

menampilkan dsta setelah penghapusan

Output :

Setelah dihapus:

0. Brio | ...

Hasil output

```
<terminated> Driver_2511533015 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.openjdk.hotsp
a = Avanza | 2020 | 1300cc | Rp150000000 | Toyota
b = Brio | 2022 | 1200cc | Rp170000000 | Honda

Data Mobil:
0. Avanza | 2020 | 1300cc | Rp150000000 | Toyota
1. Brio | 2022 | 1200cc | Rp170000000 | Honda

Setelah dihapus:
0. Brio | 2022 | 1200cc | Rp170000000 | Honda
```